Dependency & Supply Chain Inventory

CerbaSeal-Core · cerbaseal-review portal · Complete third-party inventory

Prepared in response to due diligence question on open-source and third-party dependencies

COLUMN KEY

RT = Required at runtime (the software needs this to function while running)
DEP = Required for deployment (needed to install or set up the software)
DEV = Development and testing only (never touches a client deployment)

CONTENTS

Part 1 — CerbaSeal-Core: Runtime Dependencies

What is required for the enforcement library to execute

CerbaSeal-Core has exactly ONE runtime dependency: the Node.js runtime itself. The enforcement library contains zero npm packages at runtime. All imports in the compiled output (CerbaSeal-Core/dist/) are either (a) internal module files or (b) Node.js built-in modules (node:crypto, node:path, node:fs). This is verified by inspecting every import statement in the compiled dist/ output.

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
Node.js v22.22.0	JavaScript runtime. Required to execute any CerbaSeal-Core code.	MIT / Apache-2.0 (OpenJS Foundation)	Y	Y	N	MEDIUM
[3] Client supplies this. Not shipped by Lamont Labs. Version pinned via Replit Nix module nodejs-22.						
node:crypto (built-in)	SHA-256 hashing for the audit log hash chain. Node.js standard library — no npm package.	Node.js built-in	Y	N	N	LOW
[3] Part of Node.js. No separate installation. No third-party. Used for hash-chaining, not key-based signing.						
node:http, node:fs, node:path, node:url (built-in)	Standard I/O used by the demo server. Node.js standard library — no npm packages.	Node.js built-in	Y	N	N	LOW
[3] Confirmed by browser-demo/server.ts imports: all "node:" prefixed, none are npm packages.						

Confirmed conclusion: a client deploying CerbaSeal-Core installs zero npm packages. The supply-chain surface at runtime is: Node.js (client-supplied) + CerbaSeal source code (Lamont Labs). Nothing else.

Part 2 — CerbaSeal-Core: Development & Test Dependencies

Required to run tests and build the library. Never present in a client deployment.

These packages exist in CerbaSeal-Core/node_modules/ and are used only to run the test suite (pnpm test / vitest) and build the compiled output. They are not installed in any client environment and are not shipped with the library. Evidence: CerbaSeal-Core has no package.json — these are hoisted by the workspace pnpm install.

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
vitest 2.1.9	Test runner. Executes all 372 tests and 15 audit checks. Used only via "pnpm test".	MIT	N	N	Y	LOW
I³ Located in CerbaSeal-Core/node_modules/vitest. Not present in dist/. Client never sees this.						
vite 5.4.21	Build bundler used internally by vitest for test transforms. Not used for the library build.	MIT	N	N	Y	LOW
I³ Transitive dependency of vitest. Not a direct dep. MIT licence.						
esbuild 0.21.5	JS/TS transformer used internally by vite (and vitest). Not used in library output.	MIT	N	N	Y	LOW
I³ Transitive dep of vite/vitest. Note: a different esbuild version (0.27.3) is used by tsx in the portal — these are independent installations.						
TypeScript 5.9.3	Type checker. Used at build time (tsc) to produce the compiled dist/ output and for type safety.	Apache-2.0	N	N	Y	LOW
I³ Apache-2.0 is permissive — no copyleft, no source disclosure. Dev tool only. Not in client deployment.						
chai 5.3.3	Assertion library used by vitest tests (expect(), assert()). Test-time only.	MIT	N	N	Y	LOW
I³ Transitive dep of vitest. Not a direct dep of CerbaSeal-Core.						
@types/node 22.19.17	TypeScript type definitions for Node.js built-ins. Used at compile time only — no runtime effect.	MIT	N	N	Y	LOW
I³ Type declaration package. Produces no runtime code. Apache-2.0 compatible.						

Part 3 — cerbaseal-review Portal: Runtime Dependencies

cerbaseal.replit.app — the live demo and review portal

> The cerbaseal-review portal is the demo and review environment at cerbaseal.replit.app. It is NOT a client deployment. It is a Lamont Labs-operated demonstration. The dependencies in this section are present in the portal environment only. A client deployment under Mode C would not use this stack — it would use only CerbaSeal-Core (Part 1).

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
Node.js v22.22.0	JavaScript runtime. Executes the demo portal server and all enforcement code.	MIT / Apache-2.0 (OpenJS Foundation)	Y	Y	N	MEDIUM
!^{3} Provisioned by Replit via Nix module "nodejs-22". Version pinned in .replit config.						
tsx 4.21.0	TypeScript executor. Allows running .ts files directly without a separate compile step. Used as the entry point for the portal server ("tsx examples/browser-demo/server.ts").	MIT	Y	Y	N	MEDIUM
!^{3} This is the only npm production dependency of cerbaseal-review. Declared in root package.json "dependencies". Present because the portal runs TypeScript source directly rather than pre-compiled JS.						
esbuild 0.27.3	JavaScript/TypeScript transformer. Used by tsx internally to transpile TypeScript on the fly.	MIT	Y	Y	N	MEDIUM
!^{3} Transitive dep of tsx. Declared as "esbuild": "~0.27.0" in tsx's own package.json. No direct dependency relationship from CerbaSeal-Core.						
get-tsconfig 4.13.6	Resolves tsconfig.json configuration. Used by tsx to understand TypeScript compilation settings.	MIT	Y	Y	N	LOW
!^{3} Transitive dep of tsx. Declared as "get-tsconfig": "^4.7.5" in tsx's package.json.						
resolve-pkg-maps 1.0.0	Resolves Node.js package import maps. Used by get-tsconfig.	MIT	Y	Y	N	LOW
!^{3} Transitive dep of get-tsconfig. No own dependencies. Leaf node in the dependency tree.						

PORTAL FRONTEND (BROWSER-SIDE)

The review portal's browser-facing frontend (index.html, review.html, client.js) is vanilla HTML and JavaScript. There are no frontend frameworks, no npm packages loaded in the browser, no CDN dependencies, and no external script tags. All frontend code is hand-written and served as static files directly from the server. Evidence: browser-demo/server.ts serves only readFileSync(filePath) responses with no bundling step.

Part 4 — cerbaseal-review Portal: Development & Test Dependencies

Required to test and develop the portal. Not present in any client environment.

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
TypeScript 5.9.3 *[3] devDependency in root package.json. Apache-2.0 — permissive, no copyleft.*	Type checker. Used for "tsc --noEmit" type checking pass and the compiled build.	Apache-2.0	N	N	Y	LOW
vitest 2.1.9 *[3] devDependency in root package.json. Required only to run "pnpm test".*	Test runner. Executes the full test suite including adversarial scenarios.	MIT	N	N	Y	LOW
vite 5.4.21 *[3] Transitive dep of vitest. Not present in any production code path.*	Bundler used internally by vitest. No direct use in portal production path.	MIT	N	N	Y	LOW
pdfkit 0.18.0 *[3] devDependency in root package.json. Has 6 transitive deps (see below). None reach production.*	PDF generation library. Used only by scripts/generate-*.mjs to produce report documents (the CTO binder and this document). Not imported by any enforcement or portal code.	MIT	N	N	Y	LOW
@noble/ciphers **@noble/hashes** various *[3] Transitive dep of pdfkit. @noble packages are well-audited open-source crypto. Dev-only.*	Cryptographic primitives used by pdfkit for PDF encryption features.	MIT	N	N	Y	LOW
fontkit various *[3] Transitive dep of pdfkit. Dev-only. Never touches enforcement code.*	Font processing used by pdfkit to embed fonts in generated PDFs.	MIT	N	N	Y	LOW
js-md5 / linebreak / png-js various *[3] Transitive deps of pdfkit. Dev-only. No enforcement relevance.*	Supporting utilities used by pdfkit (MD5 for PDF internals, Unicode line breaking, PNG parsing).	MIT	N	N	Y	LOW
@types/node 22.19.17 *[3] devDependency in root package.json.*	TypeScript type definitions for Node.js built-ins. Compile time only.	MIT	N	N	Y	LOW
pnpm 10.26.1 *[3] Required at install / deployment time. Not present at runtime. Open source.*	Package manager. Used to install all dependencies and run workspace scripts.	MIT	N	Y	N	LOW

Part 5 — Infrastructure & Hosting Inventory

Third-party services, platforms, and operational dependencies

This section covers non-npm third-party dependencies: hosting platforms, build infrastructure, and operational services. These are the categories most likely to be material for a client due diligence review of the portal environment specifically.

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
Replit Commercial SaaS	Cloud hosting platform for the cerbaseal-review portal (cerbaseal.replit.app). Provides compute, networking, TLS termination, and process management for the demo environment.	Commercial	Y	Y	N	HIGH
[3] The demo portal runs on Replit's US-operated infrastructure. Client data must not be sent to this environment. A client deployment under Mode C bypasses Replit entirely.						
Nix / NixOS Replit-managed	Replit uses the Nix package manager to provision the Node.js runtime (nodejs-22) and shell environment inside the Replit container. Configuration is in .replit (modules = ["nodejs-22"]).	Open source (LGPL / MIT)	Y	Y	N	LOW
[3] Managed entirely by Replit. Lamont Labs does not configure or maintain the Nix layer directly. Relevant only to the Replit demo environment.						
GitHub Commercial SaaS (Microsoft)	Source code hosting for the CerbaSeal-Core repository. Used for version control and code access.	Commercial	N	N	N	MEDIUM
[3] Repository access is gated by the proprietary licence. GitHub itself holds no operational role at runtime. Relevant for supply-chain provenance and access control review.						

EXTERNAL SERVICES CALLED AT RUNTIME

Zero external services are called at runtime by either CerbaSeal-Core or the cerbaseal-review portal server. Confirmed by source code review of all enforcement code paths and the portal server. No outbound HTTP requests, no telemetry, no analytics, no authentication providers, no message queues, no cloud storage APIs.

TLS / NETWORK

The demo portal at cerbaseal.replit.app is served over HTTPS. TLS is terminated by Replit's infrastructure. Lamont Labs does not manage certificates directly. In a Mode C client deployment, TLS would be managed entirely by the client's own infrastructure.

NPM REGISTRY

All packages are resolved from the public npm registry (registry.npmjs.org) at install time. No private registry is configured. In an air-gapped or private-registry client deployment, packages would need to be vendored or mirrored. This is architecturally straightforward and does not require code changes.

Part 6 — Files That Reference Uninstalled Packages

Packages referenced in source files but not installed

One file in the examples/ directory contains an import for a package that is not installed. This is documented for completeness and transparency.

Name & Version	Purpose	Licence	RT	DEP	DEV	Review Priority
express not installed	HTTP framework referenced in examples/http-wrapper.ts line 1. Not present in package.json, not in the pnpm lockfile, not in node_modules.	MIT (if installed)	N	N	N	LOW

[3] This file is not part of the running portal. It is an example stub. "pnpm test" and "demo:web" do not execute or import this file. Confirmed: not in pnpm store.

IMPLICATION

If examples/http-wrapper.ts were to be run, it would fail immediately with a module-not-found error. This confirms it is not part of any active code path. A client security review would note this file exists but should correctly classify it as a non-operational stub.

Part 7 — Supply Chain Risk Summary

Consolidated view across both systems

BY DEPLOYMENT TARGET

A client deployment of CerbaSeal-Core (Mode C):

Supply chain surface: Node.js runtime (client-supplied) + CerbaSeal-Core source code. Zero npm packages at runtime. Zero external services. Zero outbound network calls. A security review of a Mode C deployment needs to assess: (1) Node.js v22.x LTS and its standard library, (2) CerbaSeal source code only. That is the complete surface.

The cerbaseal-review demo portal (Replit-hosted):

Supply chain surface: Replit hosting platform + Node.js v22 + tsx 4.21.0 (MIT) + 3 transitive dependencies of tsx (esbuild 0.27.3, get-tsconfig 4.13.6, resolve-pkg-maps 1.0.0). Plus 8+ dev/test packages not present at runtime. Client data must not be sent to this environment.

LICENCE INVENTORY SUMMARY

MIT	tsx, esbuild, get-tsconfig, resolve-pkg-maps, vitest, vite, chai, @types/node, pdfkit and all its transitive deps, pnpm, resolve-pkg-maps	*No restrictions on commercial use. No source disclosure obligation.*
Apache-2.0	TypeScript (compiler / type checker only)	*Permissive. No copyleft. No source disclosure. Patent grant included.*
Node.js	Node.js runtime (MIT, Apache-2.0, and others per module)	*OpenJS Foundation. Well-governed. Standard enterprise acceptance.*
GPL / LGPL	Nix (Replit-managed environment layer only)	*Present only in Replit's hosting infrastructure. Not present in any deliverable to a client. LGPL for Nix itself; not embedded in CerbaSeal.*
Proprietary	Replit hosting platform, GitHub	*Commercial SaaS. Not part of client deployment. Demo environment only.*

KEY FACTS FOR SUPPLY CHAIN DUE DILIGENCE

1. All npm packages in the runtime path have MIT licences. No GPL. No copyleft. No source disclosure obligation for clients.
2. TypeScript (Apache-2.0) is compile-time only — it produces no runtime artefacts that reach a client.
3. The pdfkit dependency and all its transitive deps are dev-only. They generate PDF reports and are never installed in a client environment.
4. The express import in examples/http-wrapper.ts is a non-operational stub. express is not installed.
5. No external services are called at runtime by either system. Confirmed by source code review.
6. No client data should be sent to the Replit-hosted portal. That environment is US-operated.
7. A Software Bill of Materials (SBOM) in CycloneDX or SPDX format can be generated on request.
8. An air-gapped or private-registry deployment is architecturally straightforward and requires no code changes.
9. The npm registry is contacted at install time only. Not at runtime.
10. No cryptographic signing of audit records exists today. The audit log is hash-chained (SHA-256 via node:crypto). HMAC signing is a documented future requirement.

Section A — Shortest Truthful Answer Jesse Can Give On The Call

Use this if the question is asked quickly and Jesse needs a clean, precise, short answer. Every word here is accurate and defensible.

"There are two separate answers — one for the library, one for the demo.

The CerbaSeal enforcement library itself has zero runtime dependencies. When a client installs it, the only things running are Node.js — which the client already has — and CerbaSeal's own source code. Nothing else.

The live demo at cerbaseal.replit.app adds one npm package called tsx, which is MIT licensed and has three small transitive dependencies — all MIT. The demo also runs on Replit, which is a US-operated commercial hosting platform. That platform is for evaluation purposes only — client data would never go there. A client deployment would run entirely within the client's own infrastructure.

All licences are MIT or Apache-2.0. No GPL anywhere. No external services are called at runtime."

Section B — Full Technical Answer for Olivia

CERBASEAL-CORE (THE ENFORCEMENT LIBRARY)

Runtime dependencies: zero npm packages. The compiled library (CerbaSeal-Core/dist/) contains only internal module imports and Node.js built-in imports (node:crypto for SHA-256 hashing, node:http, node:fs, node:path, node:url). All confirmed by reading every import statement in the dist/ output.

Development and test only: vitest 2.1.9 (MIT), vite 5.4.21 (MIT), esbuild 0.21.5 (MIT), TypeScript 5.9.3 (Apache-2.0), chai 5.3.3 (MIT), @types/node 22.19.17 (MIT). None of these reach a client environment.

Runtime: Node.js v22.22.0 (OpenJS Foundation, MIT/Apache). Client-supplied. Version pinned to nodejs-22 LTS channel.

CERBASEAL-REVIEW PORTAL (THE DEMO)

Runtime npm dependencies: tsx 4.21.0 (MIT). This is the only item in "dependencies" in package.json. tsx has three transitive runtime dependencies: esbuild 0.27.3 (MIT), get-tsconfig 4.13.6 (MIT), resolve-pkg-maps 1.0.0 (MIT).

Reason tsx is a runtime dependency: the portal executes TypeScript source files directly via "tsx examples/browser-demo/server.ts" rather than pre-compiling to JavaScript first. In a client deployment, this would typically be replaced by a pre-compiled build, removing tsx from the runtime path.

Portal frontend: vanilla HTML and JavaScript. No frontend frameworks, no CDN, no external scripts. Served as static files.

Infrastructure: Replit commercial SaaS (US-operated). TLS managed by Replit. Node.js provisioned via Nix. This entire layer is specific to the demo environment and is not part of any client deployment.

PACKAGE NOT INSTALLED BUT REFERENCED IN SOURCE

examples/http-wrapper.ts contains "import express from 'express'" at line 1. Express is not in package.json, not in the pnpm lockfile, not in node_modules, and not in the pnpm store. This file is not executed by any running process and is not imported by any other file. It is a non-operational example stub.

CRYPTOGRAPHIC DEPENDENCIES

SHA-256: used for the hash-linked audit log chain. Implemented via Node.js built-in node:crypto. No npm package required. No third-party cryptographic library.

Important limitation: the audit log is hash-linked, not cryptographically signed. Hash chaining proves that no log entry has been modified after being written. It does not prove the identity of who created the entries. HMAC signing with a persistent key is a documented future requirement. A client security team would likely identify this gap.

Section C — What a Client Security Team Would Actually Need to Review

FOR A MODE C DEPLOYMENT OF CERBASEAL-CORE

Mode C means the client controls the hosting environment. A client security team reviewing this deployment would need to assess:

Node.js v22.x LTS

The runtime. Well-known, widely deployed, maintained by the OpenJS Foundation. Standard enterprise acceptance. Recommend pinning to a specific patch version for production.

CerbaSeal-Core source code

The enforcement library itself. All TypeScript source. Reviewable in full. No obfuscation, no compiled binary blobs. The compiled dist/ output is also available for review.

node:crypto (built-in)

SHA-256 is used for hash-chaining the audit log. Standard algorithm, standard library. The limitation (hash-linking vs. cryptographic signing) should be understood and accepted by the client before production use.

The audit log storage mechanism

Currently in-memory. Lost on restart. This must be replaced with a persistent implementation (file-backed or database-backed) before any pilot. This is the single most material security gap.

pnpm (install time only)

Used to install packages. Not present at runtime. If the client requires an air-gapped environment, packages must be vendored prior to deployment.

FOR THE DEMO PORTAL ENVIRONMENT (IF OLIVIA OR HER TEAM ACCESSES CERBASEAL.REPLIT.APP)

Additionally: tsx 4.21.0 and its 3 transitive deps (esbuild, get-tsconfig, resolve-pkg-maps), all MIT. Replit hosting platform and its infrastructure. No client data should be submitted to this environment during security review — it is US-operated SaaS.

WHAT A CLIENT SECURITY TEAM WOULD NOT NEED TO REVIEW

- vitest, vite, chai, pdfkit, TypeScript, @types/node — none of these are present in any client deployment
- express — not installed, not part of any active code path
- The pnpm lockfile entries for development packages — not relevant to client runtime risk
- Replit infrastructure details — only relevant to the demo, not a client deployment

Section D — What Exists Today vs. What Is Only the Replit Demo

EXISTS IN BOTH THE DEMO AND A CLIENT DEPLOYMENT

CerbaSeal enforcement logic	The same source code and compiled library exists in both. No separate version.
12 enforcement invariants	ALLOW / HOLD / REJECT outcomes. Identical behaviour in both environments.
Hash-linked audit log	Same implementation. In-memory in both today. Pre-pilot fix applies to both.
Evidence bundle generation	Same implementation. Produces identical output in both environments.
Node.js built-in crypto (SHA-256)	Same standard library in both.
372 tests, 15 audit checks	Test suite runs against the same source in both environments.

EXISTS ONLY IN THE REPLIT DEMO — NOT IN A CLIENT DEPLOYMENT

tsx 4.21.0 (and its 3 transitive deps)	The portal runs TypeScript source directly. A client deployment would use pre-compiled JavaScript, removing tsx from the runtime path entirely.
Replit hosting platform	US-operated commercial SaaS. Not present in any client deployment. Mode C means client controls all infrastructure.
Nix/NixOS environment layer	Replit's internal environment provisioning. Completely invisible to and absent from a client deployment.
pdfkit and report generation scripts	Dev-only tools for generating PDF documents. Not present in any client environment.
vitest, vite, chai (test suite)	Test execution environment. Not deployed anywhere.
cerbaseal.replit.app domain / TLS	Replit-managed. A client deployment would use the client's own domain and TLS infrastructure.

DOES NOT EXIST ANYWHERE YET — IN DEMO OR DEPLOYMENT

Persistent audit log storage	Currently in-memory in both environments. Required before any pilot. Estimated 2–5 days of work.
HMAC or cryptographic signing of audit records	Documented as a future requirement. Hash-linking is implemented; key-based signing is not.
Dockerfile or container build	No containerisation exists. Deployment to a client environment requires manual Node.js setup.
Deployment runbook	No step-by-step operational guide for a first deployment to an external environment.
SBOM (Software Bill of Materials)	Not generated yet. Can be produced on request.
Third-party security review	Not completed. All security review has been internal.